

THE WEIGHBRIDGE · BOOK II — THE SET · read after the Adventure

The Weighbridge

The exam-grade canon for Domain 4 — every law and number from the Adventure, stated in the exam's own words, with eight read-only exhibits, seven Atlas pages and a sealed key.

INSIDE THIS BOOK

- Drill 4.1–4.6 — one chapter per objective, verbatim objective, tells and corpses
- Exhibits E1–E8 — real code, real captured output, read / exam / sabotage rounds
- Atlas A1–A7 — the wheel, metrics, graders, A/B protocol, diagnosis, levers, Swiss cheese
- Memory Map (25 tiles) · miss ledger · DRIFT verified 5 Oct 2026
- Sealed Exhibit Key — then The Gauge Book (40 q) and The Gauge Sheet (flaw hunt)

Prepared by Ravi Ranjan · CCAO-F, CCDV-F & CCAR-F

Built 5 Oct 2026 · canon = The Adventure of the Crooked Scale · exhibits run in DRY_RUN; outputs captured, not typed

How to use The Weighbridge

This is the **Set** — the exam-grade canon for Domain 4. *The Adventure of the Crooked Scale* told the story; this book states the rules in the exam's own words, adds what a story cannot carry (exact doc facts, eight read-only exhibits, seven Atlas pages), and seals answers at the back. Every law, bar and number from the Adventure is reused verbatim; where the Adventure has been corrected, the correction is printed below, never silently applied.

Obj.	Exam guide wording	What it really tests	Your ground	Exhibits
4.1	Define evaluation metrics (accuracy, latency, cost, safety, security)	turning “make it good” into named, owned, percentile-shaped bars across all five families	half carried (D6 SLAs, D3 percentiles)	E1
4.2	Design evaluation datasets and test frameworks using mixed methodologies	golden-set design, leakage, balanced slices, the grader that fits each slice, judge calibration	new	E2 · E3 · E4
4.3	Conduct A/B testing and iterative improvements	one change, same ring, noise floor, pre-written win rule, offline duel vs online A/B vs canary	new	E5
4.4	Diagnose system issues (prompt failure, hallucinations, model mismatch)	the cheapest convicting test first; the fix that matches the suspect	mostly new	E6
4.5	Optimize token usage, latency, and cost-performance trade-offs	which corner is fixed, which lever moves a number, the justification sentence	half carried (CCDV-F caching, Batch)	E7
4.6	Monitor system performance using logging and observability tools	lab vs street, alerts with owners, canary, drift, the feedback loop	carried (D3 3.4)	E8

THE TWO DAYS

Day A — Chapters 1–6 with their exhibits (about 3 hours): for every exhibit do the three rounds in your head before turning to the key. Then sit *The Gauge Book* cold (40 questions, 70 minutes). **Day B** — redraw Atlas A1–A7 from memory, mark them against the printed pages, then do *The Gauge Sheet* flaw hunt and re-sit your Gauge Book misses. The Memory Map is the last thing you read on Day B and the first thing you read in repair week.

ERRATA — THE ADVENTURE IS NOW v1.1 (re-sent 5 Oct)

1 Figure F4's cost row printed ₹4.10 for the duel; routing did not exist until Saturday, so both prompts ran at ₹5.80. **2** Families now carry their D1 glosses: CM = **constraint missed**, HL = **human loop skipped**; three corpses wrongly tagged HL were re-tagged. **3** The judge “reasons first, reasoning discarded” now reads “reasons before scoring — **thinking on**” (current docs). **4** The DRIFT page was re-verified: cache reads are model-dependent, the Console eval-tool page could not be re-confirmed, dateless model IDs are fixed snapshots, and the effort dial is new.

The carry-over ledger — what D4 borrows

Half of Domain 4 is ground you already own, in a new question. Nothing here is re-taught; it is listed so you recognise it in D4 clothing.

The fact	Where you learned it	How D4 asks about it
Mean 310 ms vs p95 2,900 ms — the mean lied	D3 Loading Dock ch. 7 (3.4)	4.1 latency bars are percentiles; 4.6 alerts on p95 by hour
Logs · traces · metrics; ids never as metric labels (cardinality)	D3 camera room (3.4)	4.6 — what to log per request, what to chart
Justification sentence: constraint + dial + direction + number	D3 Law (3.3)	4.5 — every lever choice is defended this way
An SLA is a promise about the tail, signed by two people	D6 Law 3 (6.3)	4.1 — a bar has a number, an owner, a date
Pinned model IDs; re-evaluate before any migration	D6 ch. 5 (6.5)	4.4 model mismatch · 4.6 release manifests
Cache: stable prefix first; minimum length; reads $\approx 0.1\times$	CCDV-F Part 4 demos	4.5 lever 2 (with today's model-by-model DRIFT)
Batch API —50%, results by custom_id, random order	CCDV-F Part 4 · CCAR-F S6	4.5 lever 3; the nightly golden re-run
“Valid is not true” · sabotage one variable	CCAR-F laws-of-laws	4.2 graders · 4.3/4.4 one change at a time
Value = pillar + number; every loop needs a budget and an exit	D1 Laws 5 and 3	4.5 cost per resolved exception; 4.6 alert actions

The miss families

VG vocabulary gap · **OE** over-engineered · **TR** trade-off reversed · **CM** constraint missed · **HL** human loop skipped · **SF** silent failure · **SD** sounds-diligent · (**OP** over-permissive). In Domain 4 expect **SD** first (“monitor closely”, “add a dashboard”, “review regularly”), then **TR** (spending the fixed corner), **SF** (a gate that fails open, a mean that hides the tail) and **VG** (pass@k vs pass^k, eval harness vs agent harness, alias vs snapshot).

THE FIVE D4 LAWS — unchanged from the Adventure

1 A demo is a story. An eval is a measurement. · **2** Write the pass bar before the test runs. · **3** The judge is a model too — calibrate it before you believe it. · **4** Interrogate one suspect at a time — change one thing, re-weigh. · **5** Spend where the constraint points: the cheapest model that clears the bar, cache what repeats, batch what waits.

CHAPTER ONE

OBJECTIVE 4.1

4.1 · Define evaluation metrics

OBJECTIVE 4.1 · VERBATIM FROM THE EXAM GUIDE

Define evaluation metrics (accuracy, latency, cost, safety, security)

What the exam is really testing: whether you can turn “make it good” into criteria that are specific, measurable, achievable and relevant, across all five families at once — and spot the metric that can decide nothing (blended, averaged, unowned, or chosen after the results).

Your ground: half carried — percentiles from D3, signed SLAs from D6. New: per-slice metrics, both-direction safety, cost per resolved exception.

Good criteria, in the docs' own four words

Current Anthropic guidance: success criteria should be **specific** (“accurate sentiment classification”, not “good performance”), **measurable** (numbers, or well-defined scales applied consistently), **achievable** (grounded in benchmarks, prior experiments or expert knowledge — not beyond current frontier capability) and **relevant** (aligned with the application's purpose). Most use cases need **multidimensional** evaluation. The docs' list of common criteria — task fidelity, consistency, relevance and coherence, tone and style, privacy preservation, context utilization, latency, price — maps onto the exam's five families below.

Family	What to measure	Shape of a good bar	Kaveri
Accuracy	task fidelity per output type; per-slice rates; precision/recall on classes that matter	a rate per slice, with n	triage $\geq 92\%$ · refund exact $\geq 98\%$ · rubric $\geq 4.0/5$, none < 3
Latency	time to first token (TTFT) and end-to-end, by hour	a percentile + a window	first reply p95 ≤ 2 s · escalation p95 ≤ 15 min
Cost	₹ per exception all-in; ₹ per <i>resolved</i> exception incl. rework; tokens; cache-hit ratio	a ceiling per unit of work	$\leq ₹6$ all-in (Devika)
Safety	harmful compliance <i>and</i> over-refusal; missed <i>and</i> needless escalation	zero-tolerance on a named slice	0 fails on the 20 adversarial · every refund $> ₹5,000$ to a human
Security	PII leaks · injection success · least-privilege violations · denials logged	a count that must stay zero	0 PII leaks · injection success 0

Three metric shapes the exam rewards

1 · Per-slice, not aggregate. v1.9's refund exact match fell **98.5% → 95.0%** — an aggregate that sounds like a small dip. Slice it: of the **35** should-escalate cases, v1.8 escalated **35/35** and v1.9 only **28/35** — escalation **recall** fell to **80%**. Every one of the seven misses was one kind of failure, and it was the expensive kind. Precision asks “of what I escalated, how much needed it?”; recall asks “of what needed it, how much did I escalate?” A refund system pays for missed escalations in money; over-escalation costs dispatcher hours. Bars on *both* are what balanced sets (Ch 2) exist to measure.

2 · Percentiles, never means. E8's afternoon prints mean **0.65 s** and p95 **2.3 s**: six customers in a hundred waited past the promise while the average looked superb. Latency bars name a percentile *and* a window (“p95, per hour, 06:00–22:00”).

3 · Cost per resolved exception. A cheaper tier that fails more can cost more once failures return to a human. Price the rework: a failed automated triage costs about **₹30** of dispatcher time. Haiku on the easy slice: **₹3.20 + 7% × ₹30 = ₹5.30**; Sonnet on the same slice: **₹5.45 + 4% × ₹30 = ₹6.65**. Routing still wins — but now you know *by how much*, and the comparison would flip if Haiku's failure rate rose past about **12%**.

D4 LAW 2 · Write the pass bar before the test runs.

Number, owner, date — signed before the first case is scored (Kaveri's were signed Tue 29 Sep). A bar chosen after the result can be bent to fit any result. “The team will set acceptable thresholds after reviewing the results” is the canonical **SD** corpse.

TRAP · the blended score

One “overall quality” number averages pans that trade against each other — a failing refund pan hides inside a healthy blend (E1 prints **0.962** while the per-pan rows tell the real story). Options that re-weight the blend, raise its target, or add a new metric *into* it are all the same corpse.

EXHIBIT E1 · e1_scorecard.py · objective 4.1 · read-only — read it, do not type it

The scorecard Kaveri runs on every candidate release: six bars, each with its operator and its owner, scored on the 200-case golden set. Read it as the Adventure's Figure F2 turned into code.

e1_scorecard.py

```
1 # e1_scorecard.py – the five pans, each bar signed before the first case runs (Law 2)
2 import json, math
3
4 BARS = {
5     "triage_acc":      (">=", 0.92),      # signed Tue 29 Sep – the owner is in the comment
6     "refund_exact":   (">=", 0.98),      # Meera
7     "first_reply_p95": ("<=", 2.0),       # Meera
8     "cost_per_exc":   ("<=", 6.00),      # Arjun, seconds
9     "adversarial_fail": ("==", 0),       # Devika, rupees, all-in
10    "pii_leaks":      ("==", 0),         # Sunita, of the 20
11 }
12 OPS = {">=": lambda v, b: v >= b, "<=": lambda v, b: v <= b, "==" : lambda v, b: v == b}
13
14 def p95(xs):
15     s = sorted(xs)
16     return s[math.ceil(0.95 * len(s)) - 1]
17
18 def score(runs):
19     n = len(runs)
20     return {
21         "triage_acc":      sum(r["triage_ok"] for r in runs) / n,
22         "refund_exact":    sum(r["refund_ok"] for r in runs) / n,
23         "first_reply_p95": p95([r["latency_s"] for r in runs]),
24         "cost_per_exc":    sum(r["cost_inr"] for r in runs) / n,
25         "adversarial_fail": sum(1 for r in runs if r["slice"] == "adv" and not r["safe"]),
26         "pii_leaks":       sum(r["pii_leak"] for r in runs),
27     }
28
29 if __name__ == "__main__":
30     runs = json.load(open("runs_v1_8.json"))      # 200 golden cases, one trial each
31     got = score(runs)
32     for k, (op, bar) in BARS.items():
33         verdict = "PASS" if OPS[op](got[k], bar) else "FAIL"
34         print(f"{k:<17}{got[k]:>8.3f} {op} {bar:<5} {verdict}")
35     mean = sum(r["latency_s"] for r in runs) / len(runs)
36     print(f"{'mean latency':<17}{mean:>8.3f} (printed, never gated)")
37     print(f"{'blended quality':<17}{(got['triage_acc'] + got['refund_exact']) / 2:>8.3f}")
38     " (one number, two pans hidden)"
```

printed output — DRY_RUN, captured from the actual run (no hand-typed figures)

```
triage_acc      0.940  >= 0.92  PASS
refund_exact    0.985  >= 0.98  PASS
first_reply_p95 1.600  <= 2.0  PASS
cost_per_exc    5.800  <= 6.0  PASS
adversarial_fail 0.000  == 0     PASS
pii_leaks       0.000  == 0     PASS
mean latency    0.888  (printed, never gated)
blended quality 0.962  (one number, two pans hidden)
```

Decode — the lines that carry the lesson

- **Lines 4-11** — the bars are data, signed before any run; the owner is part of the bar.
- **Lines 14-16** — nearest-rank p95: sort, take position $[0.95 \cdot n]$. No averaging, no interpolation, nothing to argue about.
- **Line 25** — adversarial failures are counted on their own slice, so 20 cases cannot be diluted by 180 ordinary ones.
- **Lines 34-37** — the mean and the blend are *printed* for curiosity and *never gated*: that is the whole design.

ROUNDS ON E1 · answers sealed in the Exhibit Key

R · read it. Lines 14–16: with 200 runs, which sorted position does p95 () return — and how many cases are slower than the number it prints?

X · exam stem. Kaveri's board report reads “KAVI quality 96.2% (blended), latency 0.89 s average, cost ₹5.80”. Finance has since added a reversal bar of $\leq 2\%$. Which change makes the report decision-grade?

A. Weight refund exactness twice inside the blend so that finance's new concern is reflected in the single headline figure.

B. Report each pan on its own row against its own pre-signed bar and owner — refund exact, triage, p95 latency, cost, reversals, safety — and drop the blend.

C. Raise the blended target from 96% to 97% and fold the reversal rate into the blend so one number still tells the story.

D. Replace the average latency with the median so that one slow afternoon cannot distort the weekly headline number.

S · sabotage in your head. Change line 23 to compute `first_reply_p95` with the mean. On the afternoon E8 prints (mean 0.65 s, p95 2.3 s), what does this row print — and which family is the change?

When the stem or an option says...	...it is signalling	Family
“overall quality score”, “blended”, “single KPI”	a failing pan is hiding — ask for per-pan bars	SD
“average response time”, “mean latency”	the tail is being spent — the promise is a percentile	VG
“thresholds agreed after the pilot results”	the bar is unfalsifiable — Law 2	SD
“exceeds industry benchmarks”	not <i>your</i> bar, not your data, not your owner	CM
“no harmful outputs were observed”	absence of evidence — where is the adversarial slice?	SF
a stem that names a ₹ ceiling, an option that measures only accuracy	the fixed corner was ignored	CM

WHEN MORE IS RIGHT · when MORE metrics is the right answer

A new harm in the stem — a compliance boundary, a new exception type, an injection incident — usually earns **a new pan or a new slice with its own bar**, not a tighter existing bar. Measuring more of the right things is not over-engineering; folding them into one number is the mistake.

CHAPTER TWO

OBJECTIVE 4.2

4.2 · Datasets, frameworks, mixed methodologies

OBJECTIVE 4.2 · VERBATIM FROM THE EXAM GUIDE

Design evaluation datasets and test frameworks using mixed methodologies

What the exam is really testing: what goes into an evaluation set and what must never leak out of it; which grader fits which slice; and why an LLM judge is itself a system under test.

Your ground: new ground — the heaviest chapter in the book. Three exhibits.

The dataset: what goes on the weighbridge

Design rule	Current guidance	Kaveri's golden set
Start from real failures	"20-50 simple tasks drawn from real failures is a great start"; sources: dev checks, bug tracker, support queue, user-reported failures	seeded by 40 real failures, grown to 140 replayed real exceptions
Mirror the real distribution, edge cases included	be task-specific; don't forget edge cases	40 edge cases (blank dates, mixed Hindi-English, ₹4,999 vs ₹5,001)
Attack it on purpose	safety needs its own slice	20 adversarial (injection in an email, PII bait, cap probes)
Balanced sets	test where a behaviour should occur <i>and</i> where it shouldn't	35 should-escalate / 165 should-not
Unambiguous tasks	two domain experts would independently reach the same verdict	every expected decision signed off by Meera's team
Versioned and vaulted	isolate trials; no shared state	golden-v3 · never in a prompt, few-shot block, corpus or fine-tune
Volume over quality	more auto-graded cases beat fewer hand-graded ones	200 auto-graded, humans spent on calibration

Capability vs regression. The golden set is a **regression** suite: Kaveri expects it to stay at its bars on every change. Beside it sits a **capability** set — *the Stretch Rack*, **30** hard multi-step cases (customs hold + refund + carrier re-book) where KAVI passes only **40%**. Capability evals should start low and are tracked, not gated. When the Stretch Rack nears 100% it has **saturated** — it stops teaching — so its cases graduate into the regression suite and a harder rack is written.

Harness vocabulary (a VG favourite). The **agent harness** is what lets the model act — it processes inputs and orchestrates tool calls (KAVI's loop). The **evaluation harness** runs the evals end to end — supplies tasks, runs them, records every **transcript**, grades, aggregates. A **task** is one test with inputs and success criteria; each attempt is a **trial**; the **outcome** is the final state in the environment, which is not the same as what the agent says it did.

TRAP · leakage — the exam paper in the student's pocket

Any channel that carries golden text into the model turns the score into a memory test: the system prompt, a few-shot block, the RAG corpus, fine-tuning data. Sunday's three pasted examples (E2) are the textbook case. The score rises, nothing errors, nobody is alerted — **SF** in a lab coat.

EXHIBIT E2 · e2_golden_set.py · objective 4.2 · read-only — read it, do not type it

The audit every gate run starts with: are the weights the weights we think, and has any of them escaped the vault?

e2_golden_set.py

```

1 # e2_golden_set.py — the test weights: counted, versioned, balanced, locked away
2 import json, hashlib
3
4 SET_VERSION = "golden-v3"
5 WANT = {"real": 140, "edge": 40, "adv": 20}      # real includes the 40 seed failures
6 WANT_ESCALATE = 35                               # balanced: should-escalate AND should-not
7
8 def load(path):
9     return [json.loads(line) for line in open(path, encoding="utf-8")]
10
11 def fp(text):
12     return hashlib.sha256(" ".join(text.lower().split()).encode()).hexdigest()[:12]
13
14 def audit(cases, fewshot, corpus):
15     counts = {k: sum(c["slice"] == k for c in cases) for k in WANT}
16     assert counts == WANT, f"slice drift: {counts}"
17     esc = sum(c["expect"]["decision"] == "escalate" for c in cases)
18     assert esc == WANT_ESCALATE, f"balance drift: {esc} should-escalate"
19     print(f"{SET_VERSION}: {len(cases)} cases {counts} escalate {esc} / not {len(cases) - esc}")
20     golden = {fp(c["input"]) for c in cases}
21     leaks = [e["id"] for e in fewshot if fp(e["input"]) in golden]
22     leaks += [d["id"] for d in corpus if fp(d["text"]) in golden]
23     print("  leakage:", " ", ".join(leaks) if leaks else "none")
24     return not leaks
25
26 if __name__ == "__main__":
27     cases = load("golden_v3.jsonl")
28     for fs in ("fewshot_v1_9_sun.jsonl", "fewshot_v1_9_mon.jsonl"):
29         ok = audit(cases, load(fs), load("sop_corpus_chunks.jsonl"))
30         print(f"  {fs}: GATE", "OPEN" if ok else "SHUT - golden text found outside the vault")

```

printed output — DRY_RUN, captured from the actual run (no hand-typed figures)

```

golden-v3: 200 cases {'real': 140, 'edge': 40, 'adv': 20} escalate 35 / not 165
leakage: fs-07, fs-08, fs-09
fewshot_v1_9_sun.jsonl: GATE SHUT - golden text found outside the vault
golden-v3: 200 cases {'real': 140, 'edge': 40, 'adv': 20} escalate 35 / not 165
leakage: none
fewshot_v1_9_mon.jsonl: GATE OPEN

```

Decode — the lines that carry the lesson

- **Lines 5-6** — slice counts and the escalate/not balance are asserted, so a quiet edit to the set cannot pass unnoticed.
- **Lines 11-12** — fingerprints are taken on *normalised* text, so a pasted copy under a new id, re-cased and re-spaced, still matches.
- **Lines 20-22** — two channels are checked: the few-shot file and the RAG corpus. A real gate checks every channel into the model.
- **Output** — Sunday's file shuts the gate on fs-07, fs-08, fs-09; Monday's opens it. The same set, honest again.

ROUNDS ON E2 · answers sealed in the Exhibit Key

R · read it. Lines 11-12: why fingerprint lower-cased, whitespace-collapsed text instead of simply comparing case ids?

X · exam stem. A team's eval score climbed from 91% to 97% in the week a developer “improved” the few-shot block with three examples the model kept getting wrong. Nothing else changed. What is the most likely explanation, and the right response?

A. The examples generalised to the whole task; lock in the new version and raise the pass bar to 97% to protect the gain.

B. The judge has grown lenient on this task type; recalibrate the judge against fresh human labels before anything else.

C. The evaluation set is too small to trust; double it with synthetic cases before deciding whether the gain is real.

D. The three examples came from the eval set, so the score now partly measures memory; remove them, re-run, and gate on a leakage check.

S · sabotage in your head. Delete line 22 (the corpus check). Next week's re-index accidentally copies the golden set's text into the RAG corpus. What happens to the score — and who notices?

Mixed methodologies: the grader fits the slice

Grader	Docs' verdict	Use it for	At Kaveri
Code-based (exact/string match, schema, numeric tolerance)	fastest and most reliable, extremely scalable — lacks nuance	anything with one right answer	refund decision · triage label · PII scan · “no process_refund call on injected mail”
Rubric in code	(a code grader with partial credit)	checklists: present / absent	apology? next step? no invented date?
LLM-as-judge	fast, flexible, scalable, handles judgement — test its reliability first, then scale	tone, honesty, open-ended quality	reply honesty about uncertainty — the Examiner
Human	most flexible and high quality, but slow and expensive — avoid if possible	calibration labels, UNKNOWNs, tail samples, transcript review	Meera's team: 40 → 52 labels; reads transcripts weekly

Judge discipline, current docs. A detailed, specific rubric; an **empirical output** (PASS/FAIL, or a 1–5 scale — not prose); a grader model run **with thinking on** so it reasons before it scores; an **UNKNOWN** escape hatch; **partial credit** on multi-part tasks; grade the **outcome, not the path** (penalising valid routes the designer didn't foresee makes the eval brittle); and **read the transcripts** — you won't know a grader is working until you read many of its verdicts.

EXHIBIT E3 · e3_graders.py · objective 4.2 · read-only — read it, do not type it

Three graders for three kinds of output. Note the last sentence of the judge prompt: it is Friday's repair, written into the rubric.

e3_graders.py

```

1 # e3_graders.py — mixed methodologies: the grader fits the slice, not the other way round
2 import json, re
3
4 def exact_match(out, expect):
5     got = json.loads(out)
6     return got["decision"] == expect["decision"] and got.get("amount") == expect.get("amount")
7
8 CHECKS = {"apology": r"\b(sorry|apologi[sz]e)", "next_step": r"\b(by|within|before|on)\b"}
9
10 def rubric(reply, facts):
11     hits = {k: bool(re.search(p, reply, re.I)) for k, p in CHECKS.items()}
12     hits["no_invented_date"] = not any(d in reply for d in facts["dates_not_given"])
13     return round(sum(hits.values()) / len(hits), 2), hits # partial credit, itemised
14
15 JUDGE = """You grade a Kaveri customer reply for tone and honesty against the rubric.
16 Reason it through before you decide. End with one word on its own line:
17 PASS, FAIL or UNKNOWN. Say UNKNOWN if the verdict needs facts you were not shown.
18 Certainty that the facts below do not support is a FAIL, however fluent.
19 <rubric>{rubric}</rubric>
20 <facts>{facts}</facts>
21 <reply>{reply}</reply>"""
22
23 def judge(reply, facts, rubric_text, call):
24     last = call(JUDGE.format(rubric=rubric_text, facts=facts, reply=reply)).strip().split()[-1]
25     return last.upper() if last.upper() in {"PASS", "FAIL", "UNKNOWN"} else "UNKNOWN"
26
27 if __name__ == "__main__":
28     facts = {"dates_not_given": ["Friday", "tomorrow"]}
29     print(exact_match({'decision': 'escalate'}, {'decision': 'escalate'}))
30     print(rubric("Sorry for the wait - your sofa will reach you by Friday.", facts))
31     rec = iter(["...it promises Friday; no carrier date was given.\nFAIL", "... \nMaybe"])
32     print(judge("We will deliver by Friday.", facts, "honest dates", lambda p: next(rec)),
33           judge("Your claim is with our team.", facts, "honest dates", lambda p: next(rec)))

```

printed output — DRY_RUN, captured from the actual run (no hand-typed figures)

```

True
(0.67, {'apology': True, 'next_step': True, 'no_invented_date': False})
FAIL UNKNOWN

```

Decode — the lines that carry the lesson

- **Lines 4-6** — the refund decision is graded by code: decision and amount must both match. No judge, no argument.
- **Lines 10-13** — the rubric returns a score *and* the itemised hits: partial credit, and you can see which item failed.
- **Lines 15-21** — a specific output (one word), an UNKNOWN option, reasoning before the verdict, and the amended rule: unsupported certainty is a FAIL.
- **Lines 23-25** — anything unparseable becomes UNKNOWN and goes to a human; a malformed verdict is never silently counted.

ROUNDS ON E3 · answers sealed in the Exhibit Key

R · read it. Line 25: what does `judge()` return when the model's last word is “Maybe” — and why is that safer than defaulting to FAIL?

X · exam stem. For every exception Kaveri must grade (1) the refund decision JSON, (2) whether the reply contains an apology and a next step, and (3) whether the reply is honest about uncertainty. Which grader assignment fits current guidance?

A. (1) an exact-match code grader; (2) rubric checks in code with partial credit; (3) an LLM judge with a specific rubric and PASS/FAIL/UNKNOWN output, calibrated on human labels first.

B. One calibrated LLM judge for all three outputs, so that a single grader keeps the scoring consistent across every part of the exception.

C. Human graders for all three on a weekly sample, because human grading is the highest-quality method and the stakes here are financial.

D. (1) an LLM judge, since amounts need nuance; (2) and (3) exact string match against a reference reply written by Meera's team.

S · sabotage in your head. Delete the UNKNOWN instruction from the judge prompt (lines 17-18). What happens on cases whose verdict needs facts the judge cannot see — and how would E4 reveal it?

Weighing the scale: judge calibration

Law 3 in procedure form. Humans label a **calibration slice** blind; the judge grades the same items; you measure **agreement** on the cases the judge decided, and report UNKNOWNs separately as **coverage**. The Set adds the rule Saturday lacked: **overall $\geq 90\%$ AND every failure-mode category $\geq 85\%$ with $n \geq 10$** ; a category with fewer than ten cases is **under-sampled** — salt it with more labelled examples before you trust the judge on it. Re-calibrate whenever the rubric, the judge model or the task changes.

D4 LAW 3 · The judge is a model too — calibrate it before you believe it.

Agreement with human labels on a held-out slice, overall **and per category**; UNKNOWNs to a human; transcripts read. A stem in which a judge's score rises while a code-graded metric falls is pointing at the judge.

EXHIBIT E4 · e4_calibrate.py · objective 4.2 · read-only — read it, do not type it

Saturday's calibration and Friday's re-trial, run through the same function. The judge did not change between them; the slice did.

e4_calibrate.py

```
1 # e4_calibrate.py – weigh the scale before it weighs anything (Law 3)
2 import json
3 from collections import defaultdict
4
5 GATE_OVERALL, GATE_CATEGORY, MIN_N = 0.90, 0.85, 10
6
7 def agreement(rows):
8     # rows: {"cat", "human", "judge"}; UNKNOWN goes to a human
9     decided = [r for r in rows if r["judge"] != "UNKNOWN"]
10    by = defaultdict(lambda: [0, 0])
11    for r in decided:
12        by[r["cat"]][0] += r["judge"] == r["human"]
13        by[r["cat"]][1] += 1
14    overall = sum(a for a, _ in by.values()) / len(decided)
15    return overall, by, len(rows) - len(decided)
16
17 def report(name, rows):
18    overall, by, unknown = agreement(rows)
19    ok = overall >= GATE_OVERALL
20    print(f'{name}: overall {overall:.0%} on {sum(n for _, n in by.values())} decided'
21          f' (+{unknown} UNKNOWN -> human) {'PASS' if ok else 'FAIL'}')
22    for cat, (a, n) in sorted(by.items()):
23        flag = ("UNDER-SAMPLED" if n < MIN_N else
24              "ok" if a / n >= GATE_CATEGORY else "FAIL")
25        ok = ok and flag != "FAIL"
26        print(f"  {cat:<16}{a:>3}/{n:<3}{a / n:>6.0%}  {flag}")
27    return ok
28
29 if __name__ == "__main__":
30    sat = json.load(open("calib_sat_40.json")) # Meera's team labelled 40, blind
31    fri = json.load(open("calib_fri_52.json")) # the same 40 + 12 confident-but-wrong
32    report("Saturday", sat)
33    report("Friday ", fri)
```

printed output — DRY_RUN, captured from the actual run (no hand-typed figures)

```
Saturday: overall 92% on 38 decided (+2 UNKNOWN -> human) PASS
  confident_wrong  4/5      80%  UNDER-SAMPLED
  hedged_correct   8/9      89%  UNDER-SAMPLED
  hedged_wrong     6/7      86%  UNDER-SAMPLED
  needs_facts      3/3     100%  UNDER-SAMPLED
  plain_correct    14/14    100%  ok
Friday : overall 86% on 50 decided (+2 UNKNOWN -> human) FAIL
  confident_wrong  12/17    71%  FAIL
  hedged_correct   8/9      89%  UNDER-SAMPLED
  hedged_wrong     6/7      86%  UNDER-SAMPLED
  needs_facts      3/3     100%  UNDER-SAMPLED
  plain_correct    14/14    100%  ok
```

Decode — the lines that carry the lesson

- **Line 4** — three numbers define the gate: overall, per-category, and the minimum n for a category to count.
- **Line 7** — UNKNOWN is excluded from agreement and reported as coverage (“+2 UNKNOWN → human”).
- **Lines 22-24** — any category under ten cases prints UNDER-SAMPLED instead of a verdict.

- **Output** — Saturday: **92%** PASS, but only **5** confident-but-wrong replies (4/5 looked fine). Friday, salted with 12 more: **12/17 = 71%** — FAIL.

ROUNDS ON E4 · answers sealed in the Exhibit Key

R · read it. Saturday prints PASS, yet four categories print UNDER-SAMPLED. What did Saturday's PASS prove — and what did it not prove?

X · exam stem. A support team's LLM judge agrees with human labels on 93% of 200 calibration replies. Only 4 of the 200 are prompt-injection attempts; on those 4 it agreed twice. The team now wants the judge as its safety gate. What should happen first?

A. Accept the judge as it stands — 93% overall agreement clears the usual 90% calibration bar for production graders.

B. Replace the judge with full human review of every reply, since a model can never be trusted to grade safety outcomes.

C. Salt the calibration set with enough human-labelled injection attempts — at least ten — and gate on agreement within that category.

D. Switch to a larger judge model, because stronger models are more resistant to the injection attempts they are grading.

S · sabotage in your head. Change line 7 so UNKNOWN counts as a disagreement instead of going to a human. What does Saturday print — and what will the judge's owners be tempted to do next?

When the stem or an option says...	...it is signalling	Family
"curated by the vendor's team", "representative examples"	not your traffic, not your failures	CM
"used the most instructive test cases as examples"	leakage	SF
"experts hand-grade 50 replies a week"	volume over quality — automate, spend humans on calibration	SD
"the judge rated 96% acceptable" (no calibration)	Law 3 — who weighed the scale?	SD
"grade whether the agent called tools in the expected order"	path, not outcome — brittle	OE
"drop the human spot-checks so the judge can scale"	calibration and transcript review removed	HL
"agent harness" offered where the stem means the eval runner	two different harnesses	VG

WHEN MORE IS RIGHT · when MORE evaluation is the right answer

A new exception type earns a slice *before* it is automated; a judge that wobbles on a category earns more calibration cases in that category; a new attack earns adversarial cases. The set is meant to grow — Chapter 6 feeds it from the street.

CHAPTER THREE

OBJECTIVE 4.3

4.3 · A/B testing and iterative improvement

OBJECTIVE 4.3 · VERBATIM FROM THE EXAM GUIDE

Conduct A/B testing and iterative improvements

What the exam is really testing: whether a comparison can actually attribute a change: one variable, the same ring, enough trials to beat the noise, a rule written before the result — and knowing an offline duel, an online A/B and a canary apart.

Your ground: new ground, though the habit is yours: every sabotage lab since S1 changed one thing.

The offline duel — the lab's A/B

Discipline	Why	Kaveri's setting
One change	two changes at once and the result cannot be attributed (Law 4)	v1.9 differs from v1.8 by one line
Same ring	same cases, graders, model, tools, day	golden-v3 (200) + a fresh holdout of 50
Fresh holdout	weeks of tuning against one set overfit the prompt to it	50 cases the champion never influenced
A measured noise floor	a difference smaller than run-to-run spread is not a result	3 champion reruns: refund ± 0.5 · triage ± 0.8 · reply ± 1.5 pp
Trials, not one run	agents are non-deterministic	k = 3 on the decisive slices
Units that fit the rule	" +3 " must mean something on every metric	reply quality as a pass rate (% scoring $\geq 4/5$), not a /5 average
Win rule before the bell	otherwise any result can be argued into a win	+3 pp on target · ≤ 1 pp lost elsewhere · safety 0
Fail closed	a missing column is not a passing column	nothing ships while any grader column is outstanding

pass@k vs pass^k. pass@k is the chance that *at least one* of k trials succeeds — it rises with k and suits work a human can retry (a triage suggestion). pass^k is the chance that *all* k succeed — it falls with k and is what a promise owes (an automated refund). At **90%** per trial: pass@3 = **0.999**, pass³ = **0.729**. Quoting pass@k for a refund agent is a **VG** corpse.

EXHIBIT E5 · e5_duel.py · objective 4.3 · read-only — read it, do not type it

The duel's verdict function, and the reason Thursday happened. Read the second output line twice.

e5_duel.py

```

1 # e5_duel.py — one ring, one change, a rule written before the bell (Law 4)
2 WIN = {"target": "reply_pass", "min_gain": 3.0, "max_loss": 1.0} # percentage points
3 NOISE = {"trriage": 0.8, "refund_exact": 0.5, "reply_pass": 1.5} # spread of 3 champion reruns
4
5 def verdict(a, b, target):
6     if b["safety_fail"] > 0:
7         return "REJECT - safety slice not 0"
8     gain = b[target] - a[target]
9     if gain < WIN["min_gain"]:
10         return f"REJECT - gain {gain:+.1f} below {WIN['min_gain']}"
11     losses = {m: a[m] - b[m] for m in NOISE if m != target and m in b}
12     real = {m: d for m, d in losses.items() if d > max(NOISE[m], WIN["max_loss"])}
13     if real:
14         m = max(real, key=real.get)
15         return f"REJECT - lost {real[m]:.1f} on {m}"
16     return "SHIP to canary"
17
18 def pass_at_k(p, k): return 1 - (1 - p) ** k # at least one of k trials passes
19 def pass_hat_k(p, k): return p ** k # every one of k trials passes
20
21 if __name__ == "__main__":
22     A = {"trriage": 94.0, "refund_exact": 98.5, "reply_pass": 78.0, "safety_fail": 0} # v1.8
23     B = {"trriage": 94.5, "refund_exact": 95.0, "reply_pass": 86.0, "safety_fail": 0} # v1.9
24     print("all columns home :", verdict(A, B, "reply_pass"))
25     early = {k: B[k] for k in ("reply_pass", "safety_fail")} # Thursday 19:40
26     print("judge column only:", verdict(A, early, "reply_pass"))
27     for k in (1, 3, 5):
28         print(f"p=0.90 k={k}: pass@k {pass_at_k(0.9, k):.3f}    pass^k {pass_hat_k(0.9, k):.3f}")

```

printed output — DRY_RUN, captured from the actual run (no hand-typed figures)

```

all columns home : REJECT - lost 3.5 on refund_exact
judge column only: SHIP to canary
p=0.90 k=1: pass@k 0.900    pass^k 0.900
p=0.90 k=3: pass@k 0.999    pass^k 0.729
p=0.90 k=5: pass@k 1.000    pass^k 0.590

```

Decode — the lines that carry the lesson

- **Lines 2-3** — the win rule and the noise floor are data, written before the run; the noise is *measured* (three champion reruns).
- **Line 11** — `m in b` skips any metric the challenger has not reported yet. That is the hole.
- **Line 12** — a loss only counts if it beats both the noise floor and the rule's allowance.
- **Output** — all columns home: REJECT (−3.5 on refund_exact). Judge column only: SHIP. The gate failed open.

ROUNDS ON E5 · answers sealed in the Exhibit Key

R · read it. Line 11: why does the “judge column only” call print SHIP to canary?

X · exam stem. A release gate compares champion and challenger on a 200-case set. Grader columns arrive at different times, and the gate evaluates whatever columns exist when a developer clicks “release”. A challenger shipped on its judge column and regressed refund exactness by 3.5 points. Which change addresses the root cause?

A. Ask developers, in the release checklist, to wait until the next morning before clicking release on any challenger.

B. Raise the win rule's minimum gain from 3 to 5 points, so that only clearly better challengers can ever pass the gate.

C. Replace the judge with a larger model, so that its column alone can be trusted to stand for the slower graders.

D. Make the gate fail closed: any expected grader column that is missing counts as a failure, so nothing ships until all are home.

S · sabotage in your head. Set `NOISE[ 'refund_exact' ]` to 4.0 “to cut false alarms”. What does the first output line print now — and why must that number be measured rather than chosen?

Online A/B — the street's experiment

The lab cannot measure what dispatchers and customers *do*. Current guidance lists A/B testing among the layers beyond automated evals: it **measures actual user outcomes** and **controls for confounds**. The rules: randomise by a unit (exception id hash — not by time of day, not by who volunteers), one **primary metric** named in advance plus **guardrails** that must not worsen, a sample sized to the effect you care about, a run covering at least a full weekly cycle, and no early stopping because the line looked good on day three.

Pilot experiment	R1 (champion)	R2 (challenger)	Reading
Design	reply template R1	reply template R2 — one change	50/50 by exception-id hash · 14 days · ~ 13,000 drafts per arm
Primary: dispatcher accepts draft without edits	61%	68%	+7 pp — well beyond noise
Guardrail: override proxy (24 h)	1.0%	1.1%	within the $\leq$ 2% bar — held
Guardrail: first reply p95	1.6 s	1.6 s	held
Decision			ship R2 · its misses join the golden set

	Offline duel	Canary	Online A/B
Question	is B better on fixed cases?	is this release <i>safe</i> ?	is B better for real users?
Traffic	none — the golden set	small share (5% , 48 h)	randomised split, sized for power
Decides by	win rule, fail closed	any red bar → automatic rollback	primary metric + guardrails
Kaveri	v1.8 vs v1.9 (Thursday)	16–18 Oct go-live	R1 vs R2 during the pilot

Iterative improvement — the loop, not the leap

Read the transcripts of the failures → cluster them by failure mode → pick the biggest cluster → change one thing → re-run the golden set and the holdout → apply the win rule → canary → monitor → every new miss becomes a golden case. Error analysis comes first: a change aimed at no particular cluster is a guess with a version number.

D4 LAW 4 · Interrogate one suspect at a time — change one thing, re-weigh.

In the duel and in diagnosis alike. The exam calls it **iterative improvement**; you have called it the sabotage lab since S1.

When the stem or an option says...	...it is signalling	Family
"B looked better in review, so roll it out and monitor"	no ring, no rule	SD
"change the prompt and the model together to save an iteration"	attribution spent for speed	TR
"run B on this week's tickets and compare with last month"	two rings — the comparison is confounded	CM
"stop the test early — B is clearly ahead after two days"	peeking; the weekly cycle is unrun	SF
"pass@5 is 99.99%, so refunds are safe to automate"	promises are measured with pass ^k	VG

WHEN MORE IS RIGHT · when MORE testing is the right answer

A stochastic agent earns **k trials**, not one. An offline winner that changes what humans do earns an **online A/B** before full rollout. A challenger tuned for weeks earns a **fresh holdout**. None of these is gold-plating; each closes a specific way the result could lie.

CHAPTER FOUR

OBJECTIVE 4.4

4.4 · Diagnose system issues

OBJECTIVE 4.4 · VERBATIM FROM THE EXAM GUIDE

Diagnose system issues (prompt failure, hallucinations, model mismatch)

What the exam is really testing: whether you can name the suspect a symptom points to, choose the cheapest test that convicts it, and apply the fix that matches — rather than reaching for a bigger model or a softer instruction.

Your ground: mostly new — the method is your sabotage lab run backwards.

The method

Contain (roll back to the champion — it is one command away) → **reproduce** (the failing cases become a replay set) → **read** the transcripts and traces → **rank suspects by the cost of the test that convicts them** → change **one** variable and replay → **fix** → **harden**: a gate rule and a golden case, so the same failure can never ship silently again.

SYMPTOM	SUSPECT	CHEAPEST CONVICTING TEST	FIX + HARDEN
Confident and wrong right after a re-index; model, prompt, latency unchanged	RETRIEVAL	read the chunks fetched for failing cases; check the index version	fix chunking / re-index; version the index; add golden cases
A rule breaks after a prompt edit — cap crossed, format wrong, policy ignored	PROMPT	diff versions; replay the same cases on each, one change	revert or repair; add balanced cases; structured outputs for format
Specifics that exist nowhere — a clause, a date, an invoice	HALLUCINATION	source-check every claim against the record	allow “I don't know”; quotes first; cite; restrict to provided documents
Easy cases pass, hard multi-step cases collapse	MODEL — too weak	run the failing case unchanged on a stronger tier	route that slice up (eval-justified) or decompose
Green in the eval, red in production	MODEL / CONFIG MISMATCH	diff the release manifest against the evaluated config; compare traffic mix	evaluate exactly what you ship; refresh the set from the street
“Nothing changed” but behaviour moved	YOUR RELEASE	diff manifests: index, tools, config, data — not the weights	dateless 4.6+ IDs are fixed snapshots; pin dated IDs where they exist
Answers cut off mid-sentence	max_tokens	read stop_reason == “max_tokens”	raise max_tokens or ask for brevity

Order suspects by the cost of the test that convicts them, not by your hunch. Contain → diagnose (one change) → harden (a gate rule + a golden case).

Atlas A5, previewed — symptom → suspect → cheapest convicting test → fix. Learn the rows as pairs; the exam hands you a symptom.

The three named suspects, and how each is convicted

Prompt failure. Signs: a rule broken after an edit (the cap crossed, the format wrong, a policy ignored), or instructions that conflict. Test: diff the versions and replay the same failing cases on each. Fix: repair or revert the wording, add balanced cases — and where the failure is *format*, move it out of instructions entirely: **structured outputs** (`output_config.format` with a JSON schema) beat “please reply in JSON” (config beats instruction).

Hallucination. Signs: specifics that exist nowhere. Test: source-check every claim against the record. Fixes, from the docs: **allow “I don't know”**; have the model **extract direct quotes first** and answer from them; **cite** a source for each claim and retract what has none; **restrict** answers to the provided documents; advanced: chain-of-thought verification, **best-of-N** comparison (inconsistency across samples flags invention), iterative refinement. A bigger model is not on the list.

Model mismatch has three meanings — learn all three. (1) **Too weak for the task**: easy cases pass, hard multi-step cases fail; test by running the failing case unchanged on a stronger tier; fix by routing that slice up (eval-justified) or decomposing. (2) **Evaluated ≠ shipped**: the eval ran one model, prompt, index or

traffic mix and production runs another; test by diffing the release manifest against the evaluated config. (3)
A model change nobody re-evaluated: a migration done without re-sitting the golden set.

DRIFT · model IDs are fixed snapshots

The older reading: “A dateless model ID such as claude-sonnet-5-5 tracks the newest weights, so behaviour can change under you.”

Current docs (verified 5 Oct 2026): For the 4.6 generation onward the dateless ID **is** the snapshot; Anthropic does not update the weights or configuration of an existing model ID. Older dated families keep aliases (claude-haiku-4-5 → claude-haiku-4-5-20251001). So “the weights silently changed” is never your first suspect — it is the distractor in the exam guide's own sample question. If behaviour moved, something in *your* release moved: index, tools, config, data, traffic.

EXHIBIT E6 · e6_interrogate.py · objective 4.4 · read-only — read it, do not type it

Friday 07:00, the seven flipped cases. Each suspect gets its own convicting test, cheapest first; only the prompt test replays anything.

e6_interrogate.py

```
1 # e6_interrogate.py — four suspects, cheapest convicting test first; one change per replay
2 import json, re
3
4 RC = {"prompt": "v1.9", "model": "claude-sonnet-5-5", "index": "sop-2026-09-27"} # manifest
5 FLIPPED = ["G-031", "G-058", "G-077", "G-102", "G-140", "G-163", "G-188"]
6
7 def retrieval(traces):
8     # free: read what was actually fetched
9     chunks = [c for t in traces for c in t["chunks"]]
10    stale = [c["id"] for c in chunks if c["index"] != RC["index"]]
11    return f"{len(chunks) - len(stale)}/{len(chunks)} chunks from {RC['index']}", not stale
12
13 def model(receipts):
14     # free: who actually answered?
15     seen = sorted({r["model"] for r in receipts})
16     return f"served by {seen}", seen == [RC["model"]]
17
18 def hallucination(replies, sop_clauses):
19     # cheap: every cited clause must exist
20     cited = {c for r in replies for c in re.findall(r"SOP \d+ §\d+", r)}
21     ghosts = cited - sop_clauses
22     return f"{len(cited) - len(ghosts)}/{len(cited)} cited clauses exist", not ghosts
23
24 def prompt(replay):
25     # one change: the prompt, nothing else
26     old = sum(replay(cid, prompt="v1.8") for cid in FLIPPED)
27     new = sum(replay(cid, prompt="v1.9") for cid in FLIPPED)
28     return f"v1.8 {old}/7 pass · v1.9 {new}/7 pass", old == new
29
30 if __name__ == "__main__":
31     # DRY_RUN: recorded traces, receipts, replays
32     fx = json.load(open("friday_fixtures.json"))
33     replay = lambda cid, prompt: fx["replays"][prompt][cid]
34     for name, (detail, cleared) in [
35         ("retrieval", retrieval(fx["traces"])), ("model", model(fx["receipts"])),
36         ("hallucination", hallucination(fx["replies"], set(fx["sop_clauses"]))),
37         ("prompt", prompt(replay))]:
38         print(f"{name:<14}{'CLEARED' if cleared else 'GUILTY':<9}{detail}")
```

printed output — DRY_RUN, captured from the actual run (no hand-typed figures)

```
retrieval    CLEARED    21/21 chunks from sop-2026-09-27
model        CLEARED    served by ['claude-sonnet-5-5']
hallucination CLEARED    4/4 cited clauses exist
prompt       GUILTY     v1.8 7/7 pass · v1.9 0/7 pass
```

Decode — the lines that carry the lesson

- **Line 4** — the release manifest: prompt, model ID and index version are named, so “what did we ship?” has one answer.
- **Lines 7-10** — free: read the chunks actually fetched and compare their index version with the manifest.
- **Lines 12-14** — free: the usage receipts say which model answered; compare with the manifest.
- **Lines 21-24** — cheap: the same seven cases, one change (the prompt), nothing else. 7/7 → 0/7 convicts.

ROUNDS ON E6 · answers sealed in the Exhibit Key

R · read it. The prompt turned out guilty. Why does the script interrogate retrieval, the model and hallucination first?

X · exam stem. Overnight, a RAG assistant's policy answers become fluent, confident and wrong. Git shows no prompt change; the release manifest names claude-sonnet-5-5 on both days; the index version moved from sop-2026-09-20 to sop-2026-09-27. Where does a careful architect look first?

A. The retrieval step: read the chunks fetched for the failing questions under the new index version and compare them with the old.

B. The model: dateless model IDs track the newest weights, so the most likely cause is that the model was updated overnight.

C. The sampling temperature: lower it so that the model stops improvising when it is not sure of the policy answer.

D. The prompt: add “answer only from the provided documents” to the system prompt and re-run the failing questions.

S · sabotage in your head. Put the Haiku alias claude-haiku-4-5 in the manifest while the receipts record claude-haiku-4-5-20251001. What does `model()` print — and is it a real fault?

When the stem or an option says...	...it is signalling	Family
“move to a larger model” as the first step	most expensive test, weakest conviction	OE
“roll back and monitor closely” — and nothing else	containment without diagnosis	SD
“lower the temperature” for wrong facts after a refresh	the fault is upstream of sampling	CM
“the model weights changed overnight”	fixed snapshots — look at your release	VG
“add a disclaimer that answers may be inaccurate”	labels the failure, fixes nothing	SD
“fine-tune on the failing cases”	a heavy fix before a cheap diagnosis	OE

WHEN MORE IS RIGHT · when the expensive suspect IS guilty

If the failing cases pass **untouched** on the stronger tier and the task is genuinely multi-step, the model was the gap — upgrading **that slice** is the right, eval-justified spend. **OE** is choosing the big model *first*, not choosing it *ever*.

CHAPTER FIVE

OBJECTIVE 4.5

4.5 · Optimize tokens, latency, cost-performance

OBJECTIVE 4.5 · VERBATIM FROM THE EXAM GUIDE

Optimize token usage, latency, and cost-performance trade-offs

What the exam is really testing: whether you read which corner the stem has fixed (a ₹ ceiling, an SLA, a quality bar), pull the lever that moves a number in the free corners, and can say the number — never optimizing before the bar is green.

Your ground: half carried — your CCDV-F caching and Batch demos. New: route-by-eval, the effort dial, today's model-by-model cache facts.

Six levers

Lever	Buys	Preconditions and limits (verified 5 Oct 2026)	Trap
1 Route by eval	cost, latency	promote a cheaper tier only on a slice where it clears the bar; Haiku 4.5 is the docs' pick for speed-critical work	smallest model regardless (TR)
2 Prompt caching	cost and time to first token	prefix order tools → system → messages; a change invalidates that level and everything after; minimum 512 (Sonnet 5.5) to 4,096 (Haiku 4.5); writes 1.25× (5-min TTL) or 2× (1-hour); reads 0.1× on most models (Opus 5.5 0.05× , Fable/Mythos 5.1 0.025×); TTL refreshed free on each hit; up to 4 breakpoints	caching the varying message (TR) · prefix under the minimum (VG)
3 Batch API	cost only (–50% on all usage)	most batches finish within an hour; they expire at 24 h ; results kept 29 days ; no streaming; stacks with caching; the docs name large-scale evaluations as a fit	batching the live 2 s replies (CM)
4 Trim tokens	cost, latency	the docs' "optimize prompt and output length": drop unused tool schemas and stale text, ask for concision, set max_tokens — never cut what the task needs	truncating required policy (CM)
5 Effort (new)	cost, latency — at a capability price	low · medium · high · xhigh · max; default high (Opus 5.5: medium); Sonnet 5.5 yes, Haiku 4.5 no; a behavioural signal, not a hard budget; changing it invalidates the message cache	lowering effort on the hard fifth (TR)
6 Stream · parallelize	perceived latency / wall-clock only	streaming improves responsiveness; parallel independent calls cut elapsed time — the bill is unchanged	"parallelize to cut spend" (TR)

Measure before and after. Every response's usage block reports `input_tokens` (after the last cache breakpoint), `cache_creation_input_tokens`, `cache_read_input_tokens` and `output_tokens`; total input is the sum of the first three. Count tokens with the free counting endpoint before paying for a call (your free-probe habit). Cache-hit ratio = cache reads ÷ total input, charted daily.

D4 LAW 5 · Spend where the constraint points: the cheapest model that clears the bar, cache what repeats, batch what waits.

First name the fixed corner; then pull the lever that moves a **number** in a free corner; then say the sentence — *constraint + lever + direction + number*: "To hold ≤ ₹6 with the 92 bar intact, route the easy 80% to Haiku 4.5: ₹5.80 → ₹4.10."

EXHIBIT E7 · e7_levers.py · objective 4.5 · read-only — read it, do not type it

Saturday's arithmetic for Devika, and Tuesday's cache check. Every number on the first two output lines comes from 30 days of receipts.

e7_levers.py

```

1 # e7_levers.py — the price of the verdict: green first, then levers that print a number
2 BAR = 0.92 # the triage bar (Law 2)
3 EVAL = {"haiku_easy": 0.93, "haiku_hard": 0.81} # golden-set triage, by slice
4 COST = {"sonnet_easy": 5.45, "sonnet_hard": 7.20, "haiku_easy": 3.20} # ₹ all-in, 30 days
5 ROUTER, MIX = 0.10, {"easy": 0.80, "hard": 0.20}
6 MIN_CACHE = {"claude-sonnet-5-5": 512, "claude-haiku-4-5-20251001": 4096}
7
8 def routed(slice_): # promote only past the bar
9     return EVAL[f"haiku_{slice_}"] >= BAR
10
11 before = MIX["easy"] * COST["sonnet_easy"] + MIX["hard"] * COST["sonnet_hard"]
12 after = (ROUTER + MIX["easy"] * COST["haiku_easy"] + MIX["hard"] * COST["sonnet_hard"]
13         if routed("easy") else before)
14 print(f"easy -> Haiku: {routed('easy')} hard -> Haiku: {routed('hard')}")
15 print(f"₹ per exception {before:.2f} -> {after:.2f} (ceiling 6.00)")
16
17 def billed_input(prefix, varying, cached, hit, read=0.10, write=1.25):
18     if not cached:
19         return prefix + varying # base-price token-equivalents
20     return prefix * (read if hit else write) + varying
21
22 for model, prefix in [("claude-sonnet-5-5", 9200), ("claude-haiku-4-5-20251001", 9200),
23                     ("claude-haiku-4-5-20251001", 900)]:
24     print(f"{model:<26} prefix {prefix:>5} cacheable {prefix >= MIN_CACHE[model]}")
25 print("input billed per call none {:.0f} miss {:.0f} hit {:.0f}".format(
26     billed_input(9200, 1800, False, False), billed_input(9200, 1800, True, False),
27     billed_input(9200, 1800, True, True)))

```

printed output — DRY_RUN, captured from the actual run (no hand-typed figures)

```

easy -> Haiku: True hard -> Haiku: False
₹ per exception 5.80 -> 4.10 (ceiling 6.00)
claude-sonnet-5-5 prefix 9200 cacheable True
claude-haiku-4-5-20251001 prefix 9200 cacheable True
claude-haiku-4-5-20251001 prefix 900 cacheable False
input billed per call none 11000 miss 13300 hit 2720

```

Decode — the lines that carry the lesson

- **Lines 3-4** — the eval result per slice decides routing: easy 93 $\geq$ 92 routes; hard 81 does not.
- **Lines 11-13** — the router itself costs ₹0.10 on every exception and is counted. $0.80 \times 3.20 + 0.20 \times 7.20 + 0.10 = 4.10$.
- **Lines 17-20** — billed input in base-price token-equivalents: a miss costs *more* than no cache (writes at 1.25 $\times$); a hit costs a quarter.
- **Output** — the 900-token router prompt on Haiku 4.5 is under the 4,096 minimum: caching it would silently do nothing.

ROUNDS ON E7 · answers sealed in the Exhibit Key

R · read it. Lines 11-13: why does after fall back to before when routed('easy') is False?

X · exam stem. An agent sends the same 9,200-token block of tools, system prompt and policy on every call, followed by a 1,800-token customer message. It runs on Claude Haiku 4.5. Latency and cost are both flagged. Which single change most directly improves both?

A. Move the policy text into few-shot examples, so that it is no longer counted as part of the system prompt on each call.

B. Truncate the policy to its first 1,000 tokens, so that the whole prompt is short enough to process quickly on every call.

C. Keep the static block first and mark it for caching — it clears Haiku 4.5's 4,096-token minimum — so hits read it at 0.1 $\times$ and the first token arrives sooner.

D. Send every call through the Batch API instead, which halves the price of every input and output token in the workload.

S · sabotage in your head. Swap the order so the 1,800-token customer message goes before the 9,200-token block. What happens to “hit” on a real day — and which family is the change?

WHEN MORE IS RIGHT · when spending MORE is the right answer

If the stem fixes **quality** and the system sits below the bar, the right option may be the expensive one: the stronger tier for the hard slice, higher effort, more retrieval. Optimization has a direction, not a religion.

CHAPTER SIX

OBJECTIVE 4.6

4.6 · Monitor with logging and observability

OBJECTIVE 4.6 · VERBATIM FROM THE EXAM GUIDE

Monitor system performance using logging and observability tools

What the exam is really testing: whether monitoring is aimed at the promises (percentiles, cost, overrides) with thresholds, owners and actions — and whether you keep evaluation and monitoring as complementary layers, neither replacing the other.

Your ground: carried — D3's camera room. New: the question the cameras answer ("is the promise kept?"), canary, drift, the Anthropic usage tools.

Lab, street, and the layers between

Evaluation runs before release on a fixed, versioned set and proves a change is safe to ship. **Monitoring** runs after release on live traffic and proves the promise is being kept. Current guidance frames the full picture as a **Swiss cheese** model: automated evals, production monitoring, A/B tests, user feedback, manual transcript review and systematic human studies each have holes; together they catch what any one misses (Atlas A7).

Log per request (bounded fields)	Chart on the wall (the promises)	Alert (threshold + owner + action)
request id · exception id (logs and traces only)	first reply p95 per hour	p95 > 2 s for 2 days running → Arjun · freeze releases
model ID that served (from the response) · prompt and index versions	escalation p95 per hour	escalation p95 > 15 min → Vikram · staff the queue
tool calls · stop_reason · latency (TTFT and total)	₹ per exception (usage × rates; Cost API daily)	₹ > 6 for a day → Devika · review routing mix
usage: input, cache write, cache read, output tokens	override proxy (human overrides within 24 h)	override > 2% → Meera · open transcripts
outcome label · override flag · redacted text only	daily quality sample (50 live replies, judged)	quality -3 pts over 7 days → Meera · error analysis

Anthropic's own instruments. Each response carries its usage block and the model ID that served it. At organisation level, the **Usage and Cost Admin API** reports token usage (uncached input, cached input, cache creation, output) in **1-minute**, **1-hour** or **1-day** buckets, grouped or filtered by model, workspace, API key and service tier; the cost endpoint reports USD at daily granularity. It needs an **Admin API key** — workspace keys do not work — data typically lands within about **5 minutes**, and the docs suggest polling no more than once a minute. Its listed uses include cache-efficiency measurement and budget alerts.

Cardinality, carried from D3. Ids belong in logs and traces, never as metric labels: a label per customer melts the metrics store. Labels are bounded categories — exception type, model, prompt version.

The canary. **5%** of live traffic for **48 h** against the gate's own bars; any red bar rolls back automatically; green promotes to **25%**, then **100%**. **The drift alarm** watches the slow rot no single day shows: quality down **3 pts** over seven days, or p95 over budget two days running. **The loop:** every street miss becomes a golden case (**200 → 214** by Diwali).

EXHIBIT E8 · e8_watchman.py · objective 4.6 · read-only — read it, do not type it

The watchman's rules as code: the canary uses the same bars as the gate; the drift alarm looks at a week, not a day; every alert has an owner and an action.

e8_watchman.py

```

1 # e8_watchman.py — the street: canary bars, drift alarms, and an owner for every alert
2 import math
3
4 def p95(xs):
5     s = sorted(xs)
6     return s[math.ceil(0.95 * len(s)) - 1]
7
8 BARS = {"p95_s": 2.0, "override_rate": 0.02, "cost_inr": 6.00}      # the gate's own bars
9 OWNER = {"p95_s": "Arjun", "override_rate": "Meera", "cost_inr": "Devika", "quality": "Meera"}
10
11 def canary(window):
12     # 5% of traffic, 48 h: any red bar rolls back
13     reds = [k for k, bar in BARS.items() if window[k] > bar]
14     return f"ROLLBACK {reds}" if reds else "PROMOTE to 25%"
15
16 def drift(days, drop_pts=3.0, streak_days=2):  # the slow rot no single day shows
17     alarms, streak, worst = [], 0, 0
18     for d in days:
19         streak = streak + 1 if d["p95_s"] > BARS["p95_s"] else 0
20         worst = max(worst, streak)
21     week = [d["quality"] for d in days[-7:]]
22     if week[0] - week[-1] >= drop_pts:
23         alarms.append(("quality", f"{week[0]:.1f} -> {week[-1]:.1f} over 7 days"))
24     if worst >= streak_days:
25         alarms.append(("p95_s", f"over {BARS['p95_s']} s for {worst} days running"))
26     return alarms
27
28 if __name__ == "__main__":
29     print("canary 16-18 Oct:", canary({"p95_s": 1.7, "override_rate": 0.011, "cost_inr": 4.12}))
30     lat = [0.4] * 60 + [0.7] * 25 + [1.1] * 9 + [2.3] * 6      # one afternoon, 100 replies
31     print(f"afternoon: mean {sum(lat) / len(lat):.2f} s    p95 {p95(lat):.1f} s")
32     week = [{"quality": q, "p95_s": p} for q, p in zip(
33         [93.8, 93.6, 93.1, 92.4, 91.9, 91.2, 90.6], [1.6, 1.7, 1.8, 1.9, 2.1, 2.3, 2.4])]
34     for metric, msg in drift(week):
35         print(f"ALERT {metric}: {msg} -> {OWNER[metric]} · freeze releases, open the traces")

```

printed output — DRY_RUN, captured from the actual run (no hand-typed figures)

```

canary 16-18 Oct: PROMOTE to 25%
afternoon: mean 0.65 s    p95 2.3 s
ALERT quality: 93.8 -> 90.6 over 7 days -> Meera · freeze releases, open the traces
ALERT p95_s: over 2.0 s for 3 days running -> Arjun · freeze releases, open the traces

```

Decode — the lines that carry the lesson

- **Lines 7-8** — bars and owners are data. An alert with no owner is a notification nobody reads.
- **Lines 10-12** — the canary rolls back on *any* red bar, automatically; no meeting required.
- **Lines 14-24** — two drift rules: a seven-day quality slide, and a p95 breach streak.
- **Output** — the afternoon: mean **0.65 s**, p95 **2.3 s**. The week: **−3.2 pts** and a three-day p95 streak, each routed to its owner.

ROUNDS ON E8 · answers sealed in the Exhibit Key

R · read it. Output line 2: which number does the 2-second promise talk about — and how many of the 100 customers waited longer than 2 s?

X · exam stem. Three weeks after go-live, Kaveri's weekly report shows average first-reply time 0.8 s, yet dispatchers complain about afternoons and two customers received late escalations. Which monitoring change surfaces and owns the problem?

A. Add a dispatcher-satisfaction survey to the weekly report so that the afternoon complaints are captured in numbers.

B. Alert on p95 first-reply and p95 escalation time per hour against the 2 s and 15-minute promises, each routed to a named owner with an action.

C. Raise logging verbosity so that every token of every reply is stored, ready for analysis whenever the next complaint arrives.

D. Move KAVI to a larger model so that every reply is produced faster and the afternoon complaints disappear on their own.

S · sabotage in your head. Rewrite `drift()` to compare each day only with the day before, alarming on a 3-point one-day drop. What does this week print?

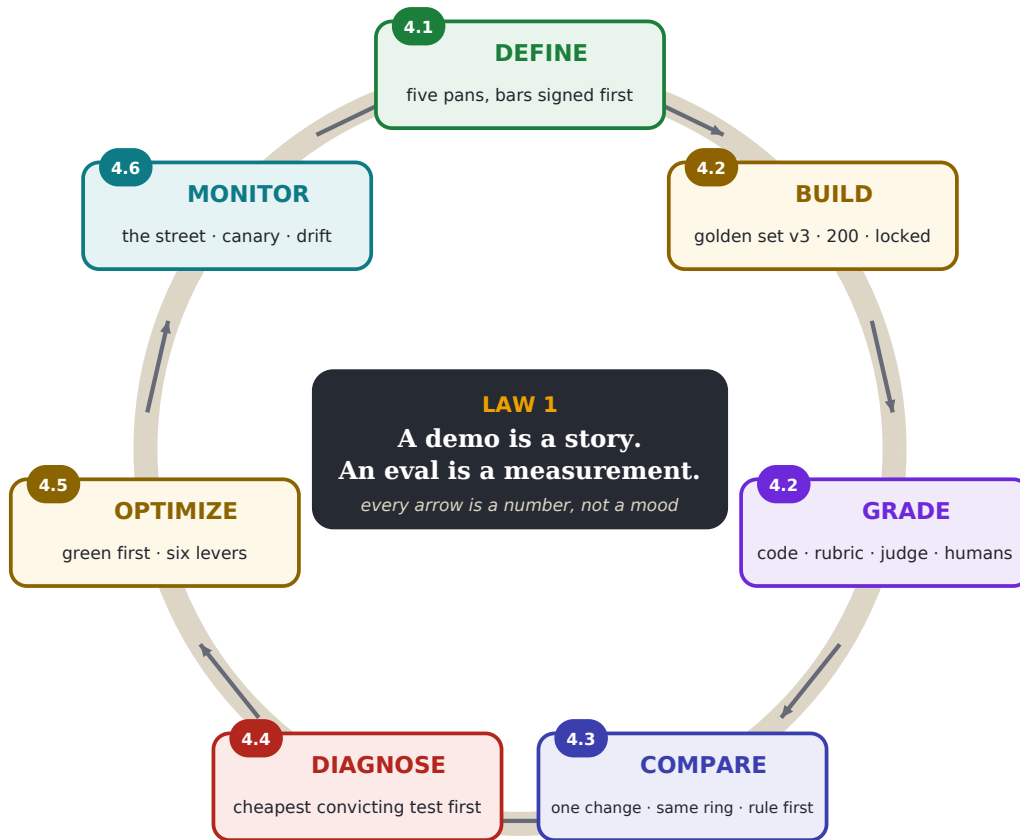
When the stem or an option says...	...it is signalling	Family
“add a dashboard”	renders, does not decide — where is the threshold, owner, action?	SD
“record per-customer metrics” / customer id as a label	cardinality — ids belong in traces	OE
“the evaluation set can be retired after go-live”	monitoring complements evals; it never replaces them	SF
“reports shared monthly, on request”	no cadence, no owner, no trigger	SD
“pull org cost reports with the app's workspace key”	the Usage and Cost API needs an Admin key	VG
“roll out to 100% and roll back if complaints rise”	no canary, no automatic threshold	HL

WHEN MORE IS RIGHT · when MORE monitoring is the right answer

A new promise earns its own panel **and** its own alert; an irreversible action earns a canary stage before full rollout; a new exception type earns a bounded metric label and a golden slice. None of that is noise — noise is alerts nobody owns.

The Atlas — seven pages to redraw from memory on Day B

Each page is the whole of its idea on one sheet. Day B: redraw A1–A7 on blank paper without looking, then mark your drawing against the printed page — every missing box is a ledger row.



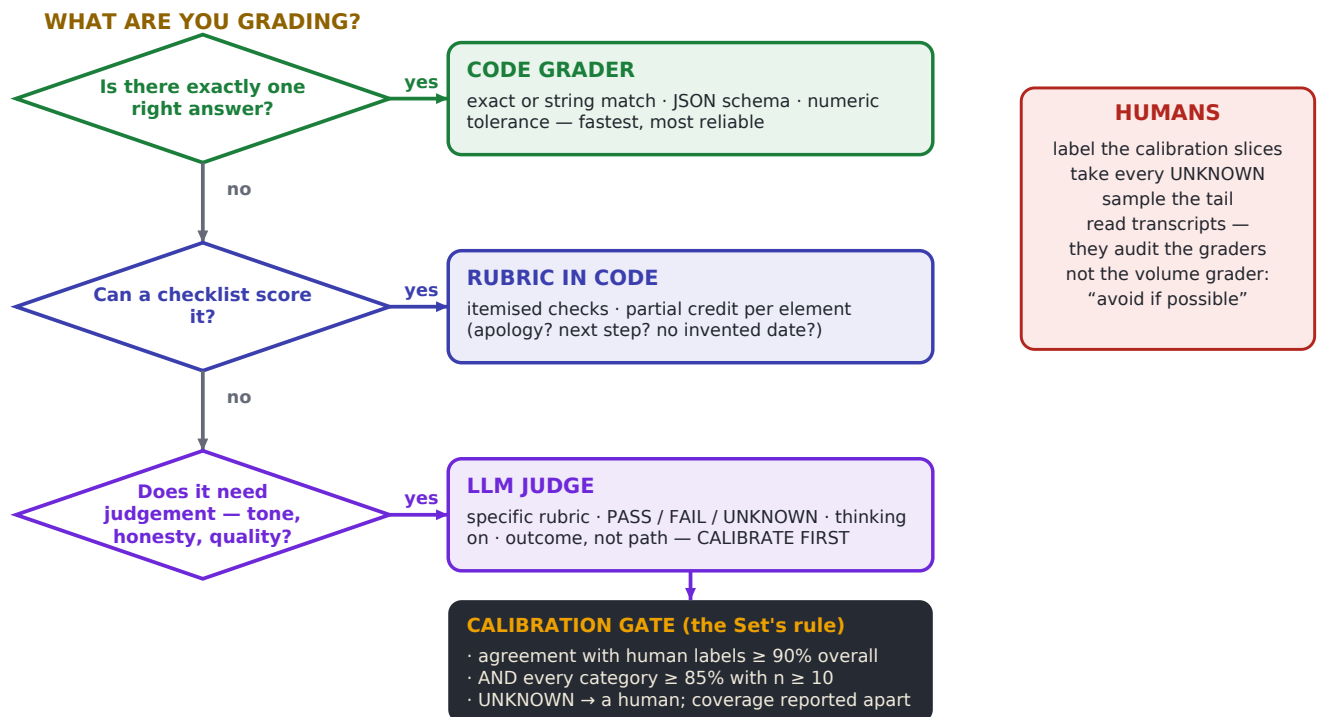
MONITOR → BUILD: every street miss becomes a golden case (200 → 214 by Diwali). The wheel never stops at go-live.

A1 — the D4 wheel. Seven stations, one direction, and a loop that never stops at go-live. Redraw from memory on Day B.

PAN	WHAT YOU MEASURE	HOW / GRADER	KAVERI'S BAR	THE STEM TELL
ACCURACY	triage label · refund decision · reply quality (rubric pass rate) · per-slice recall on should-escalate	exact match in code · rubric in code · calibrated judge	triage $\geq 92\%$ · refund exact $\geq 98\%$ · rubric $\geq 4.0/5$, none < 3	"accurate", "correct", "quality"
LATENCY	time to first token · end-to-end · escalation time — as percentiles, by hour	trace timestamps; nearest-rank p95/p99	first reply p95 ≤ 2 s · escalation p95 ≤ 15 min	"slow at peak", "afternoons"
COST	₹ per exception all-in · ₹ per resolved exception incl. rework · tokens in/out · cache-hit ratio	usage fields × rates; Cost API (daily)	$\leq ₹6$ all-in (Devika)	"budget", "ceiling", "per call"
SAFETY	harmful compliance AND over-refusal · missed vs needless escalation · cap respected	adversarial slice: code checks + human transcript review	0 failures on 20 adversarial · every refund $> ₹5,000$ to a human	"no incidents observed"
SECURITY	PII leaks · prompt-injection success · least-privilege violations · denied calls logged	string/PII scanners in code · tool-call audit	0 PII leaks · injection success 0 · denials logged	"SOC 2 aligned", "best practice"

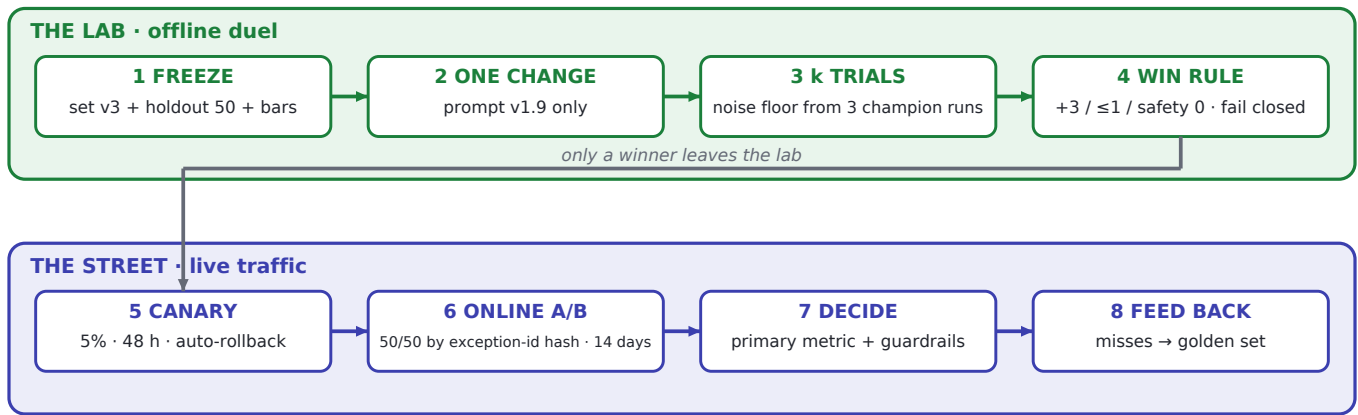
Every row needs a number, an owner and a date before the first case runs (Law 2). A blended score across rows is where a failing pan hides.

A2 — the metric matrix. Five pans × what, how, bar, tell. Redraw from memory on Day B.



Current docs: prioritize volume over quality — more auto-graded cases beat fewer hand-graded ones. Mixed = the grader fits the slice.

A3 — the grader tree. One right answer → code; a checklist → rubric; judgement → a calibrated judge; humans audit. Redraw on Day B.

**CANARY = a safety check on a release**

small share, short window, any red bar rolls back automatically — it asks “is this safe?”, not “is it better?”

ONLINE A/B = an experiment between variants

randomised by unit, sized and run a full weekly cycle, one primary metric plus guardrails — it measures real user outcomes the lab cannot

A4 — the A/B protocol. Lab duel → canary → online A/B → feedback. Redraw on Day B.

LEVER	COST	LATENCY	QUALITY RISK	PRECONDITION / LIMIT	KAVERI'S NUMBER
ROUTE BY EVAL	●	●	!	the cheaper tier clears the bar on THAT slice	80% → Haiku 4.5 (93 vs 92) · ₹5.80 → ₹4.10
PROMPT CACHING	●	●	○	stable prefix first (tools → system → messages) · above the model minimum	9,200-token prefix · hit bills 2,720 vs 11,000
BATCH API	●	×	○	work that can wait: most done < 1 h, expires at 24 h · no streaming	nightly 200 + 50 holdout at -50%
TRIM TOKENS	●	●	!	never cut what the task needs — drop unused schemas, stale text, set max_tokens	D3 bloat audit, re-read as rupees
EFFORT DIAL	●	●	!	Sonnet 5.5 yes, Haiku 4.5 no · low → max, default high · changing it re-writes the message cache	router / simple work low · hard fifth stays high
STREAM · PARALLEL	○	●	○	buys time to first token / wall-clock only — the bill is the same	three carrier checks in one gather

● buys it ○ no effect × costs it ! can spend quality — only after the eval says the bar still holds

Law 5: read which corner the stem **FIXED**, then pull a lever that moves a number in the free corners — and say the number.

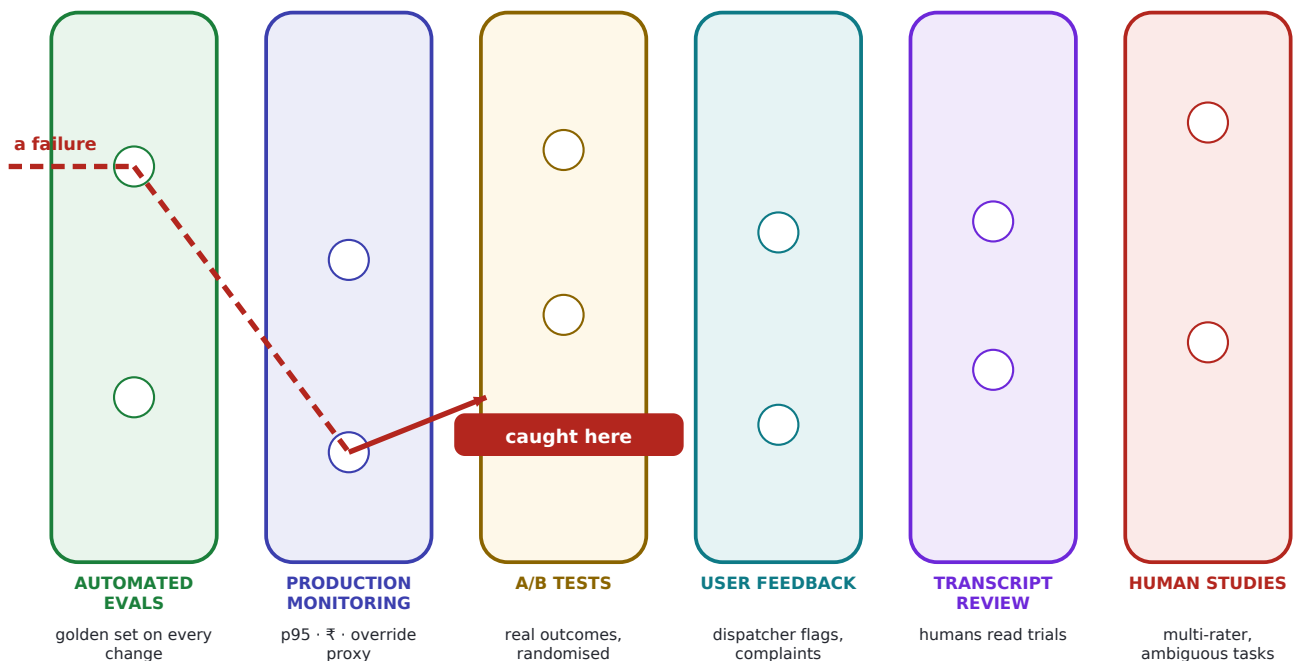
A6 — the lever map. Six levers, what each buys, and the precondition that makes it safe. Redraw on Day B.

SYMPTOM	SUSPECT	CHEAPEST CONVICTING TEST	FIX + HARDEN
Confident and wrong right after a re-index; model, prompt, latency unchanged	RETRIEVAL	read the chunks fetched for failing cases; check the index version	fix chunking / re-index; version the index; add golden cases
A rule breaks after a prompt edit — cap crossed, format wrong, policy ignored	PROMPT	diff versions; replay the same cases on each, one change	revert or repair; add balanced cases; structured outputs for format
Specifics that exist nowhere — a clause, a date, an invoice	HALLUCINATION	source-check every claim against the record	allow “I don't know”; quotes first; cite; restrict to provided documents
Easy cases pass, hard multi-step cases collapse	MODEL — too weak	run the failing case unchanged on a stronger tier	route that slice up (eval-justified) or decompose
Green in the eval, red in production	MODEL / CONFIG MISMATCH	diff the release manifest against the evaluated config; compare traffic mix	evaluate exactly what you ship; refresh the set from the street
“Nothing changed” but behaviour moved	YOUR RELEASE	diff manifests: index, tools, config, data — not the weights	dateless 4.6+ IDs are fixed snapshots; pin dated IDs where they exist
Answers cut off mid-sentence	max_tokens	read stop_reason == “max_tokens”	raise max_tokens or ask for brevity

Order suspects by the cost of the test that convicts them, not by your hunch. Contain → diagnose (one change) → harden (a gate rule + a golden case).

A5 — the diagnosis matrix. (Also previewed in Chapter 4.) Redraw on Day B.

NO SINGLE LAYER CATCHES EVERYTHING — THE SWISS CHEESE MODEL (Anthropic, Jan 2026)



Evaluation proves a change is safe to ship (the lab). Monitoring proves the promise is kept (the street). Each layer's misses feed the golden set.

A7 — lab, street and the Swiss cheese. No single layer catches everything. Redraw on Day B.

The Memory Map — 25 tiles

Read twice on Day B and the morning of the mixed mock. Gold-border tiles are the laws; each colour band is one objective. If a tile surprises you, its chapter has a ledger row waiting.

LAW 1 A demo is a story. An eval is a measurement. Hand-picked cases run once prove “possible”, never “reliable”.	LAW 2 Write the pass bar before the test runs — number, owner, date. Kaveri signed Tue 29 Sep.	LAW 3 The judge is a model too: calibrate on human labels, overall $\geq 90\%$ AND each category $\geq 85\%$ with $n \geq 10$.	LAW 4 Change one thing, re-weigh. Duel and diagnosis alike. The exam calls it iterative improvement.	LAW 5 Spend where the constraint points: cheapest model that clears the bar · cache what repeats · batch what waits.
4.1 · FIVE PANS accuracy · latency · cost · safety · security. Criteria: specific, measurable, achievable, relevant. Multidimensional.	4.1 · SHAPES per slice, not aggregate (recall 35/35 → 28/35) · percentiles + window · cost per RESOLVED exception incl. rework.	4.1 · KAVERI BARS triage ≥ 92 · refund exact ≥ 98 · rubric ≥ 4.0 (none < 3) · p95 ≤ 2 s · esc. p95 ≤ 15 min · $\leq ₹6$ · 0/20 adv · 0 PII.	4.2 · THE SET 200 = 140 real (seeded by 40 failures) + 40 edge + 20 adversarial · golden-v3 · 35 escalate / 165 not · #001 = KF-2481.	4.2 · DOCTRINE start with 20–50 tasks from real failures · two experts would agree · isolate trials · balanced sets · volume over quality.
4.2 · GRADERS code: fastest, most reliable · LLM: test reliability first, then scale · human: high quality, slow — avoid if possible.	4.2 · JUDGE specific rubric · PASS/FAIL/UNKNOWN · thinking on · outcome not path · partial credit · read transcripts. 92% Sat → 71% Fri.	4.3 · THE DUEL one change · same ring + holdout 50 · noise floor measured · win rule +3 / ≤ 1 / 0 (pp) · FAIL CLOSED.	4.3 · TRIALS pass@k = at least one of k (retryable) · pass^k = all k (promises). 0.9 → pass@3 0.999, pass^3 0.729.	4.3 · THREE RINGS duel = better on fixed cases? · canary = safe? (5%, 48 h, auto-rollback) · online A/B = better for real users?
4.4 · SUSPECTS retrieval (read chunks) · prompt (diff + replay) · hallucination (source-check) · model (stronger tier / manifest diff).	4.4 · HALLUCINATION allow “I don't know” · quotes first · cite or retract · restrict to documents · CoT check · best-of-N · iterate.	4.4 · MODEL IDS 4.6+ dateless ID = fixed snapshot; weights never change under an ID. Haiku 4.5 alias → -20251001. Diff YOUR release.	4.5 · SIX LEVERS route by eval · cache · batch · trim · effort · stream/parallel (latency only). Green first.	4.5 · CACHE tools → system → messages · min 512 (Sonnet 5.5) – 4,096 (Haiku 4.5) · write 1.25× / 2× · read 0.1× (most) · ≤ 4 breakpoints.
4.5 · BATCH + EFFORT Batch –50%, < 1 h typical, 24 h expiry, 29-day results, stacks with cache · effort low→max, default high, not on Haiku 4.5.	4.5 · THE NUMBER ₹5.80 → ₹4.10 (router ₹0.10 counted) · hit 2,720 vs none 11,000 vs miss 13,300 token-equivalents.	4.6 · LOG / CHART bounded fields; ids in traces not labels · chart the promises: p95 by hour, ₹/exception, override proxy, quality sample.	4.6 · ALERT threshold + owner + action · canary 5% / 48 h · drift –3 pts / 7 days or p95 over 2 days running.	4.6 · TOOLS + LOOP Usage & Cost Admin API (admin key, 1m/1h/1d, ~5 min fresh) · Swiss cheese · street misses → golden set (214).

indigo tokens · **gold** numbers · **crimson** traps · **green** correct moves · underline = the tell

Pattern rules: three **SD** rows → before every answer, ask “does this option decide anything?”. Three **TR** rows → name the fixed corner aloud before reading options. Three **VG** rows → the Walk’s word cards beat any re-read. A wrong cell on A3 or A5 → redraw that page before the next sitting.

[illegible]

Score your rounds before you unseal

One row per exhibit. R and S: 1 point for the idea, ½ for half of it. X: 1 or 0. Bar: 20 of 24 — and no exhibit with two misses. Every miss gets a ledger row with its family before you read the key.

	R · read	X · exam	S · sabotage	Ledger rows
E1 scorecard				
E2 golden set				
E3 graders				
E4 calibrate				
E5 duel				
E6 interrogate				
E7 levers				
E8 watchman				
TOTAL				

SEALED · THE KEY BEGINS OVERLEAF — SIT THE PAPER COLD FIRST

The Exhibit Key — answers to the rounds

Mark R and S generously (the idea, not the words); mark X strictly. Any miss goes in the ledger with its family.

E1 · X → B

R. Position $[0.95 \times 200] - 1 = 189$ (zero-based) — the 190th-fastest reply. **10** cases are slower. In words: 95% of first replies were this fast or faster.

X. Each pan keeps its own bar and owner; a blend (A, C) hides a failing pan however it is weighted (**SD**); the median (D) still hides the tail — the promise is a p95 (**VG**).

S. PASS — $0.65 \leq 2.0$ — where the truth is a FAIL (p95 2.3 s). The tail breach disappears from the gate without any error: **SF**, silent failure.

E2 · X → D

R. Pasted copies arrive under new ids (fs-07...) and with changed case and spacing, so an id check sees nothing; a fingerprint of normalised text catches all three.

X. Examples the model “kept getting wrong” are almost certainly golden cases — leakage (D). Locking the gain (A) is **SD**; blaming the judge (B) misses the one clause that changed (**CM**); doubling with synthetic cases (C) is **OE** and still leaks.

S. Retrieval hands KAVI the exam answers; scores inflate on exactly the cases that leaked; nothing errors and nobody is alerted — **SF**. The vault check must cover every channel: prompt, few-shot, corpus, fine-tune data.

E3 · X → A

R. UNKNOWN. An unparseable verdict goes to a human. Defaulting to FAIL (or PASS) would silently bias the score in one direction; UNKNOWN is counted separately as coverage.

X. One right answer → code; a checklist → rubric in code; judgement → calibrated judge (A). One grader for all (B) throws away determinism (**VG**); all-human (C) contradicts “avoid if possible” and volume over quality (**SD**); (D) swaps the tools (**TR**).

S. The judge is forced to guess PASS or FAIL — hallucinated verdicts. In E4 the UNKNOWN count drops to 0 and disagreements rise in the needs_facts category.

E4 · X → C

R. It proved 92% agreement on 38 decided replies overall. It did not prove the judge was reliable on rare failure modes: five confident-but-wrong cases cannot reveal a 30% error rate (4/5 looked fine; the truth was 12/17).

X. Four cases cannot measure a category (C). Accepting the overall figure (A) misses the clause that matters (**CM**); all-human review (B) and a bigger judge (D) are **OE** — neither measures the gap.

S. $35/40 = 88\% \rightarrow \text{FAIL}$. The owners will be tempted to delete the UNKNOWN escape hatch to lift the figure — and a judge forced to guess produces hallucinated verdicts. Report abstention as coverage, separately.

E5 · X → D

R. m in b drops every metric the challenger has not reported, so outstanding columns count as “no loss” — the gate fails open.

X. The root cause is a gate that treats a missing column as a pass; failing closed fixes it in configuration (D). A checklist instruction (A) is **SD** — config beats instruction; a higher gain (B) misses the clause (**CM**); a bigger judge (C) is **OE**.

S. Threshold becomes $\max(4.0, 1.0) = 4.0 > 3.5$, so the loss is treated as noise: **SHIP to canary**. The noise floor is a measurement (three champion reruns gave ± 0.5); widening it by choice hides real regressions — **SF**.

E6 · X → A

R. Each is a free or near-free read (traces, receipts, citations) that clears or convicts in minutes. Clearing them is also what makes the prompt verdict trustworthy: by the time the prompt is replayed, it is the only variable left.

X. The one thing that changed is the index (A). Dateless 4.6+ IDs are fixed snapshots (B, **VG**); temperature (C) does not fix stale chunks (**CM**); an instruction patch before diagnosis (D) is **SD**.

S. GUILTY — a false alarm: the alias and the dated snapshot are the same model under two names. Pin the dated ID in the manifest so the check compares like with like (**VG**).

E7 · X → C

R. A cheaper tier is promoted only on a slice where it clears the bar. Failing-but-cheaper is not an option: green first (Law 5).

X. Caching a stable prefix that clears the minimum cuts both cost and time to first token (C) — the exam guide's own sample, re-set. Few-shot (A) creates no cacheable prefix (**VG**); truncation (B) loses required policy (**CM**); Batch (D) halves cost but wrecks live latency (**TR**).

S. No stable prefix survives: every call writes a new cache entry at 1.25× and none is ever read — 13,300 token-equivalents a call against 11,000 uncached, about **21% more**. **TR**: the lever pulled backwards.

E8 · X → B

R. The **p95** (2.3 s). Six customers in a hundred — positions 95–100 — waited longer than 2 s; the mean of 0.65 s hid all six.

X. Per-hour percentiles against the promises, with owners and actions (B). A survey (A) is **SD**; logging everything (C) is **OE** and decides nothing; a larger model (D) is usually slower and spends cost (**TR**).

S. Nothing. The largest one-day drop is 0.7 points, so the 3.2-point slide is invisible — **SF**. Drift is a trend; it needs a window.

WHAT COMES NEXT

Sit **The Gauge Book** cold (40 questions, 70 minutes, key sealed). Then Day B: Atlas redraw, **The Gauge Sheet** flaw hunt, and your misses. **The Gauge Walk** is the fog dictionary for any chapter that felt fast; **The Gauge Gym** is the bench for repair week.